Namo Roadmap

Date: 20260511

Design philosophy

Analytical freedom

Namo's foundational design principle is that every key in a row/record/hash is a dimension. There is no distinction between 'axes' and 'values'. This means coordinates are values and values are coordinates.

Namo infers everything from the hash keys. There is no configuration step, no reshaping, no schema declaration. An array of hashes from a database, a CSV, a JSON API, or a YAML file goes straight into Namo and every key is immediately selectable, projectable, and usable in formulae.

This simplification is powerful because it eliminates the distinction that every other tool requires to be made at ingestion time. Analytical decisions belong to the analyst, to be made later during analysis. Every other library in this space requires the user to commit to analytical decisions at ingestion time — which columns are the index, which are data variables, what types they have, when computation runs, where derived values come from. Each of those commitments narrows what the user can later do with the loaded data.

The structure of an analysis — which dimensions are grouped by, which are aggregated over, which serve as identifiers and which as measurements — are determined by the questions asked of the data, not the data as structured. The same dataset answers different questions, and Namo doesn't presage any at ingestion.

A hash `{symbol: 'BHP', date: '2025-01-01', close: 42.5}` has three dimensions, all equally first-class. You can select on `close` the same way you select on `symbol` — `namo[close: 42.5]` is just as valid as `namo[symbol: 'BHP']`. You can project to `symbol` and `date` and drop `close`, or project to `close` alone. No dimension is privileged. No field is "just data."

`coordinates.close` and `values.close` are the same data viewed two ways — `coordinates` is `values` deduplicated. The shallow API distinction reflects a deep equivalence: every value is a coordinate of its dimension, every coordinate is a value present in the data. Whether you treat `close` as an axis to group by or a quantity to compute over is decided at query time.

Other libraries force the decision earlier. Pandas requires one to choose which columns are the index. xarray requires one to declare dimensions, coordinates, and data variables separately. In each case, the user must understand their data's analytical structure before they can load it — and once loaded, restructuring is awkward.

Type agnosticism

The kinds of computation you'll do are also deferred. Namo doesn't privilege numbers. A formula can concatenate strings, produce status labels, or generate alert messages as naturally as it can compute ratios:

```
e.label  = proc{ "#{symbol} (#{exchange})" }
e.status = proc{ volume > 0 ? 'active' : 'suspended' }
e.alert  = proc{ "#{symbol}: #{action} at #{close}" }
```

These are just as valid as `proc{ close / book_value }`. The formula mechanism resolves names and calls procs. What the proc does with the values — arithmetic, string manipulation, date logic, pattern matching — is Ruby's business, not Namo's.

This means Namo can handle datasets that aren't numerical at all: customer records, event logs, text corpora, legal documents with categorised metadata, survey responses. Anything expressible as an array of hashes. The selection, projection, contraction, composition, and set operators all work on text, dates, booleans, and arbitrary objects the same way they work on numbers. `namo[status: 'active']` works because `status` is a dimension like any other, and strings are values like any other.

No other tool in this space offers this. Pandas comes closest — DataFrames can hold mixed types — but its computation model (`df['col'].rolling().mean()`) assumes numeric columns. xarray is explicitly numerical. Quantrix and Improv are spreadsheets. Polars is a columnar computation engine optimised for numeric and string operations but not arbitrary Ruby objects. Namo's formula mechanism works on whatever Ruby can work on, which is everything.

Namo is not a computation engine or a multidimensional spreadsheet. It is a multidimensional database. The distinction matters: spreadsheets and numerical tools like Quantrix, Improv, xarray, Pandas, and Polars are fundamentally organised around numbers. Their dimensions exist to label numerical data. Text in a dimension is a category label, not something you compute on.

Live computation objects

Similarly to how Namo treats all dimensions as both data and coordinates, formulae are treated as derived dimensions. `row.close` and `row.earnings_yield` resolve through the same mechanism. The consumer doesn't need to know or care whether a dimension is stored data or a computed formula. They're all just names with values.

A Namo holds data and computation together as a single object whose computed values are always current with respect to its data. Stored values and formulae are accessed through one interface; both reflect current state on every access.

`row.close` returns the stored value. `row.return` invokes the proc and returns the computed value. Same syntax, same semantics from the consumer's perspective. The fact that one is stored and one is computed is an implementation detail of the dimension, not a fact about how it's accessed.

When the underlying data changes — new rows append, existing rows are updated, the backing store delivers different content — every computed value reflects the change without reconciliation. Pull-based reactivity through laziness: each access recomputes from current state, so there's no stale-snapshot problem to solve.

This holds across the formula trajectory: single-row formulae shipped in 0.1.0, cross-row formulae planned in 0.11.0, parameterised formulae in 0.12.0. Same mechanism throughout, same currentness property regardless of formula complexity.

Other tabular libraries produce snapshots. Pandas, Polars, dplyr, and DataFrames.jl all materialise computed values into stored columns, severing them from the computation that produced them. Spreadsheets like Excel and Quantrix attach formulae to cells but operate on them through a different interface than data. Namo treats them identically across the entire algebra of operations it provides.

Unified treatment of stored and derived dimensions

Following from live computation: every operation Namo provides treats stored dimensions and derived dimensions equivalently. Selection, projection, contraction, composition, set operators, comparison operators, pattern-matching against schemas — none of them distinguish how a dimension's values are produced.

```
namo[:revenue]                          # projection - works for stored or derived
namo[revenue: 1000..2000]               # selection - works for stored or derived
namo - returned_items                   # set operation - formulae carry through
namo * fundamentals                     # composition - formulae merge by rule
case other; when namo.dimensions; ...   # pattern-match - covers both kinds
```

Analytical structure can be added or removed without re-ingesting. A user can attach `:return`, `:sma_20`, and `:signal` formulae to an existing Namo at any time; subsequent queries treat them as first-class dimensions. They can be removed equally freely. The data layer is untouched; the analytical layer grows and shrinks at the analyst's discretion.

This is what "manipulating derived dimensions as data" means: not a metaphor, but a description of the architecture. Liveness is the mechanism; unified interface is the consequence.

Self-contained formulae

For the unified interface to work, formulae need to be self-contained — pure functions of the row (or `(row, namo)` for cross-row formulae). They depend on their inputs, not on the environment they were defined in.

```
# Good - pure function of row
e[:return] = proc{|row| row[:close] / row[:previous_close] - 1}

# Bad - depends on external state
@discount_rate = 0.05
e[:discounted] = proc{|row| row[:value] / (1 + @discount_rate)}
```

The self-contained pattern is what makes formulae portable. A Namo can be serialised, sent to another process or another machine, and its formulae will produce the same results because they don't depend on the environment they were defined in. The pattern is also what makes formulae safe to compose — two Namos can be merged by `*` and their formulae coexist because neither references anything outside its inputs.

This isn't enforced by the language (Ruby procs can capture whatever they like), but it's the convention Namo's design depends on. Future versions may move toward stricter enforcement or toward formula representations that don't allow external capture at all.

Algebraic operations on whole Namos

A Namo is a value in an algebra of operations, not just a collection to iterate. Set operators (`+`, `-`, `&`, `|`, `^`) combine Namos as sets of rows. Composition operators (`*`, `**`, `/`) combine Namos with different dimensions. Comparison operators (`==`, `<`, `<=`, `>`, `>=`) ask structural questions about subset relations. `===` asks pattern-match questions about analytical shape.

```
combined = ohlcv * fundamentals    # join on shared dimensions
held     = portfolio & universe    # rows present in both
losers   = today - yesterday       # rows new today
```

These operators take Namos as inputs and produce Namos as outputs. The result of every operator is itself queryable, composable, and operable. The algebra closes.

The shape of an analytical pipeline is not pre-committed to. Operators compose in whatever order makes sense for the question being asked. No other DataFrame library exposes its operations as algebraic operators. Pandas has methods (`merge`, `concat`, `drop_duplicates`); R has function syntax (`bind_rows()`, `inner_join()`); Polars has fluent method chains. Namo treats the operations as first-class operators on the language level, which makes pipelines compact, readable, and uniform.

Composition with surrounding systems

A Namo is passive. It holds state and answers queries. It does not provide subscription mechanisms, event propagation, dependency tracking, or change-detection infrastructure. The reactivity model is the analyst's decision, not Namo's.

Real-time systems built on Namo wire push-based change-detection to the Namo's mutation side and pull-based consumers to the Namo's query side. The Namo doesn't need to know about either; the surrounding system composes them.

```
# Push side: external mechanism updates the Namo
websocket.on_tick{|tick| analysis.append(tick); notifier.broadcast(:updated)}

# Pull side: consumer queries when notified
notifier.subscribe(:updated){|_| update_dashboard(analysis[criteria])}
```

Three components, three responsibilities, clean interfaces. The same passive Namo works in batch reports (pull only, no notifier), interactive analysis (manual queries), polling systems (pull on a timer), and real-time dashboards (push triggers, pull queries) without modification.

Reactive frameworks like RxJS push values through subscription graphs. Namo provides the live computation property that makes such frameworks valuable, but leaves the reactivity infrastructure to the user — to be chosen and composed with whatever the rest of the system already uses. The Namo's job is to be a current-valued data container; everything else is composable around it.

Portability of analytical artefacts

Following from self-contained formulae and the unified interface: a Namo is a portable analytical artefact. Data plus computation plus structure travel together. Serialise a Namo, send it to a colleague, and they have everything needed to query it, extend it, and recompute its derived values.

This is the property notebooks try to provide and fail at. Jupyter notebooks intermix code cells, output cells, and environment state — the artefact you receive is not deterministically re-runnable. A Namo is different: the data is structurally present, the formulae are self-contained procs, and the receiver can call any query operation on it without depending on a particular Ruby environment, package set, or execution order.

This isn't yet a shipped feature — serialisation lands later in 1.x — but the design enables it. Every other principle in this section contributes: self-contained formulae are portable; unified interface means the receiver doesn't need to know what's stored vs computed; type agnosticism means the format doesn't have to encode complex type systems; analytical structure at query time means the structure is in the data itself, not in a separate schema.

A Namo is a small, complete, self-describing analytical object. Pandas DataFrame plus the script that produced its computed columns. Excel workbook plus the ability to be queried programmatically. Jupyter notebook minus the bullshit.

Current state: 0.6.0

0.0.0 (2026-03-15): Initial release

Instantiation from an array of hashes. Namo infers dimension names from hash keys and extracts unique coordinate values per dimension automatically. Selection via keyword arguments in `[]`, supporting single values, arrays, and ranges.

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

sales.dimensions    # => [:product, :quarter, :price, :quantity]
sales.coordinates   # => {product: ['Widget', 'Gadget'], quarter: ['Q1', 'Q2'], ...}
sales[product: 'Widget']           # single value
sales[quarter: ['Q1', 'Q2']]      # array
sales[price: 10.0..20.0]          # range
```

This release established the foundational insight: every key is a dimension, every value is a coordinate. No configuration, no schema, no reshaping step.

0.1.0 (2026-03-28): Formulae and projection

Formulae via `[]=`. A formula is a proc assigned to a name. It receives a Row object and resolves named references against row data or other formulae. Formulae compose — a formula can reference another formula, and the dependency chain resolves lazily through Row.

Projection via positional symbol arguments in `[]`. Selection and projection can be combined in a single call or chained.

```
sales[:revenue] = proc{|r| r[:price] * r[:quantity]}
sales[:profit] = proc{|r| r[:revenue] - r[:cost]}

sales[:product, :quarter, :revenue]           # projection
sales[:product, :revenue, product: 'Widget']  # projection + selection
sales[product: 'Widget'][:revenue]            # chained
```

Also introduced `to_a` for extracting data as an array of hashes, the `Row` class for formula resolution, and `attr_accessor :data, :formulae`.

0.2.0 (2026-04-14): Enumerable

Namo includes Enumerable. The `each` method yields Row objects (not raw hashes), so Enumerable methods like `map`, `select`, `reduce`, `min_by`, and `flat_map` all see formulae as though they were data.

```
sales[:revenue] = proc{|r| r[:price] * r[:quantity]}
total = sales.reduce(0){|sum, row| sum + row[:revenue]}
cheapest = sales.min_by{|row| row[:price]}
```

Selection logic moved from Namo to `Row#match?`. Added `Row#to_h`. `each` returns an Enumerator when no block is given.

0.3.0 (2026-04-15): Contraction

Contraction via negated symbols in `[]`. The unary minus on a symbol (`-:price`) produces a `NegatedDimension`, which `[]` recognises as "remove this dimension from the result." The complement of projection — projection says what to keep, contraction says what to remove.

```
sales[-:price, -:quantity]                          # remove dimensions
sales[-:price, -:quantity, product: 'Widget']       # contraction + selection
```

Raises `ArgumentError` when mixing projection and contraction in the same call. Formulae carry through contraction. `NegatedDimension` and `Symbol#-@` introduced as supporting classes.

Row extracted to its own file (`Namo/Row.rb`). Tests split into per-class files.

0.4.0 (2026-04-15): Concatenation and row removal

The first two row-axis set operators.

`+` concatenates two Namos with matching dimensions. Appends rows from the second to the first. Formulae merge — self's formulae take priority on conflict.

`-` removes exact matching rows. Whole-row set difference. If a row appears in the right operand, it's removed from the left. Both operators raise `ArgumentError` when dimensions differ.

```
jan_sales + feb_sales        # more rows, same dimensions
all_sales - returned_items   # fewer rows, same dimensions
```

0.5.0 (2026-04-16): Intersection, union, symmetric difference

The remaining row-axis set operators, completing the set algebra.

`&` returns rows present in both operands. `|` returns all rows deduplicated (implemented via `Array#|`). `^` returns rows in one operand but not both — the symmetric difference.

```
sales & promotions      # rows in both
sales | new_arrivals    # all rows, no duplicates
sales ^ competitor      # rows unique to each side
```

All three require matching dimensions, raise `ArgumentError` otherwise. Formulae carry through from self, merge from other, self wins on conflict. `|` and `^` merge formulae from both sides.

0.6.0 (2026-05-11): Equality, pattern-match, subset, and superset

Equality, pattern-match, and subset/superset operators, multiset-theoretic on rows where row-content matters. Two Namos with the same rows in different orders are equal; two Namos where one's rows are contained in the other are in a subset relation regardless of how either was ordered. Duplicate rows count — `[{a: 1}, {a: 1}]` is not equal to `[{a: 1}]`. This follows from Namo's identity as a multidimensional database rather than a sequence: row order is an accident of ingestion, not a property of the data, but row count is data.

The equality hierarchy mirrors Ruby's standard convention, extended with `===` for pattern-match dispatch:

```
a.equal?(b)   # same object (inherited from BasicObject, not overridden)
a.eql?(b)     # same class + multiset-equal data + same formula names
a == b        # multiset-equal data, any class, formula names ignored
a === b       # same dimensions + same formula names, any class, data ignored
```

`eql?` requires class match because subclassing carries included modules (`TradingAnalysis < Namo` includes `Indicators`, `Scoring`); two instances of the same subclass with the same data and formulae are interchangeable as analyses, while a subclass and a bare `Namo` with the same data are not. `==` ignores class and formulae — two Namos are equal if they hold the same data, mirroring Ruby's `1 == 1.0` / `1.eql?(1.0)` convention.

`===` answers a structurally different question: does this candidate fit this analytical pattern? Same dimensions, same formula names, regardless of the rows. This is what case statements need — pattern dispatch on analytical shape, not on data identity:

```
template = Namo.new([{x: 0}])

case Namo.new([{x: 5}, {x: 6}])
when template then :matched
else :not_matched
end
# => :matched
```

`===` returns false (rather than raising) for non-Namo operands, since case statements require it to be safe to call on any value. Without a Namo-specific `===`, Ruby's default would delegate to `==`, which would dispatch case statements on multiset row equality — structurally misleading when users want analytical-type dispatch.

```
namo_a = Namo.new([{symbol: 'BHP', close: 42.5}, {symbol: 'RIO', close: 118.3}])
namo_b = Namo.new([{symbol: 'RIO', close: 118.3}, {symbol: 'BHP', close: 42.5}])

namo_a == namo_b      # true - same rows, different order
namo_a.eql?(namo_b)   # true - same class, same rows, no formulae either side
namo_a.equal?(namo_b) # false - different objects
```

`hash` is content-based and consistent with `eql?` — computed from the canonical form of `@data` (rows sorted lexicographically), the class, and the formula names. Two Namos that are `eql?` produce the same hash, making Namos usable as Hash keys and Set members.

Subset and superset follow stdlib `Set`'s precedent, generalised to multisets:

```
a < b     # a's rows are a strict multiset subset of b's rows
a <= b    # a's rows are a multiset subset of b's rows
a > b     # a's rows are a strict multiset superset of b's rows
a >= b    # a's rows are a multiset superset of b's rows
```

These pair with the set operators algebraically: `a & b == a` iff `a <= b`; `a | b == b` iff `a <= b`; `a - b ==` iff `a <= b`. Dimension mismatch raises `ArgumentError`; non-Namo operand raises `TypeError`.

The error message format for dimension mismatch was standardised to `"dimensions don't match: X vs Y"` across all binary operators (`+`, `-`, `&`, `|`, `^`, `<`, `<=`, `>`, `>=`), retrofitted into the set operators from 0.4.0–0.5.0.

Deliberately not included: `<=>` and `Comparable` (subset/superset is a partial order, not total — two Namos can be incomparable, and stdlib `Set` omits `<=>` for the same reason); Row-level public operators; block forms (relaxed-matching variants land in 0.10.0 alongside block forms for set and composition operators). Aspect-level `===` template-matching on `Namo::Dimensions`, `Namo::Coordinates`, and `Namo::Values` lands in 0.7.0.

Summary

The set operators (`+`, `-`, `&`, `|`, `^`) together with the comparison operators (`==`, `===`, `eql?`, `<`, `<=`, `>`, `>=`), selection, projection, contraction, and formulae give Namo a complete vocabulary for working with a single dataset or combining datasets that share the same dimensions. The next phase (0.7.0+) extends this to datasets with different dimensions via composition operators and adds richer selection and formula capabilities.

0.7.0: Aspect classes and values

Promote `dimensions`, `coordinates`, and `values` to first-class aspect objects via subclasses of plain Ruby types. Each aspect overrides `===` to template-match against whole Namos, enabling case-statement dispatch on schema. `values` is introduced in this release as the third aspect. `to_h` is exposed as the Ruby-conventional alias for `values`.

The conceptual unit is "aspects as first-class objects." Comparison vocabulary in 0.6.0 covered Namo-level operators; this release covers aspect-level `===` template-matching, which closes out the comparison story.

values[:dimension]

Extract all values for a dimension, preserving duplicates and row order.

```
namo = Namo.new([
  {symbol: 'BHP', close: 42.5},
  {symbol: 'RIO', close: 118.3},
  {symbol: 'BHP', close: 43.1},
])

namo.coordinates[:symbol]  # => ['BHP', 'RIO']
namo.values[:symbol]       # => ['BHP', 'RIO', 'BHP']
```

`coordinates` answers "what exists along this axis?" — the unique axis labels. `values` answers "what does this column actually contain?" — the raw data, preserving count and order. The difference is analogous to `DISTINCT` vs a bare `SELECT` on a column.

The two aspects are syntactically parallel — both `coordinates[:dim]` and `values[:dim]` use `[]` access on the returned object — and semantically parallel as well (both project the Namo down to a per-dimension Array, ready for Ruby's built-in Array operators).

Implementation:

```
def values
  @values ||= Namo::Values.new.tap do |hash|
    dimensions.each{|d| hash[d] = @data.map{|row| row[d]}}
  end
end

class Namo::Values < Hash
  # === overridden for template-matching against Namos (see Aspect classes below)
end
```

`Namo::Values` is a subclass of `Hash`, so all Hash operations work — `[]`, `keys`, `values` (the Hash method, not the Namo aspect), `to_a`, `==`, iteration. The subclass exists so that `===` can be overridden to match against whole Namos in case-statement contexts. For day-to-day use, treat the return value as a Hash; the subclass identity is only relevant for template matching.

`values[:symbol]` is plain Hash indexing. The whole hash is built up front rather than lazily per column; for typical Namo sizes (hundreds to low thousands of rows, handful of dimensions) the cost is negligible and matches the pattern `coordinates` already uses.

`values` is the primitive that `coordinates` is built on — `coordinates[:dim]` is `values[:dim].uniq`. The implementation reflects that directly:

```
def coordinates
  @coordinates ||= Namo::Coordinates.new.tap do |hash|
    dimensions.each{|d| hash[d] = values[d].uniq}
  end
end
```

A single source of truth for per-dimension column extraction. The performance cost compared to a single-pass implementation is negligible for typical data sizes (hundreds to low thousands of rows) — one extra method dispatch and array allocation per dimension — and 1.x is about expressivity and stability, not speed. The optimised form can return as a 3.x concern alongside columnar storage, when performance work is on the table.

`dimensions` is also wrapped, returning a `Namo::Dimensions` (subclass of `Array`):

```
def dimensions
  @dimensions ||= Namo::Dimensions.new(@data.first.keys)
end
```

The three introspection methods now form a complete set: `dimensions` tells you what names exist, `coordinates` tells you the unique values per dimension, `values` tells you all values for a dimension. All three return subclasses of plain Ruby types (Array, Hash, Hash), so existing usage is unchanged — the wrappers add behaviour without removing any.

`to_h` is `values`

The hash returned by `values` is exactly the columnar form expected by `to_h` — `{symbol: [...], close: [...], ...}`, dimension-keyed arrays preserving row order. They produce identical output, so `to_h` is the standard Ruby coercion idiom for the same thing:

```
def to_h
  values
end
```

Both are public, both available from 0.7.0. `values` is the primitive name; `to_h` is the Ruby-conventional name. Users can reach for either depending on context — `values` reads naturally when you're treating it as an inspection method (`namo.values[:close].sum`), `to_h` reads naturally when you're converting (`hash = namo.to_h; hash.keys`).

This is purely about output. Internal storage stays row-oriented (`@data` as `Array<Hash>`) until 3.x, where columnar storage becomes an option for performance. The public `to_h`/`values` API is stable from 0.7.0; what 3.x changes is the cost of computing the result, not the result itself.

The pairing of `values` with proc-based selection is about user workflow: if you're writing `namo[pe: ->(v){ v && v < 15 }]`, you probably also want `namo.values[:pe]` to inspect the range, spot nils, and understand the distribution before writing the predicate. Nothing in the implementation depends on this pairing — proc-based selection works through `Row#match?` and never needs the full column extracted.

Aspect classes and template-matching

The aspect-returning methods (`dimensions`, `coordinates`, `values`) return subclass instances rather than plain Array or Hash. The subclasses are:

```
class Namo::Dimensions < Array
  def ===(candidate)
    case candidate
    when Namo
      all?{|d| candidate.dimensions.include?(d)}
    else
      super
    end
  end
end

class Namo::Coordinates < Hash
  def ===(candidate)
    case candidate
    when Namo
      keys.all?{|d| candidate.dimensions.include?(d)}
    else
      super
    end
  end
end

class Namo::Values < Hash
  def ===(candidate)
    case candidate
    when Namo
      keys.all?{|d| candidate.dimensions.include?(d)}
    else
      super
    end
  end
end
```

Each subclass overrides `===` to template-match against a Namo on the right side. The aspect on the left is the template; the Namo on the right is the candidate; the question is "does this Namo conform to this aspect?"

For non-Namo right operands, `===` falls through to `super` — meaning `Array#===` for `Namo::Dimensions` and `Hash#===` for the others, which both delegate to `==`. This preserves the standard Ruby behaviour: `a.dimensions === [:symbol, :date]` does array equality, `a.coordinates === some_hash` does hash equality.

The Namo-right-operand case enables case-statement dispatch on schema:

```
case incoming
when ohlcv.dimensions
  process_ohlcv(incoming)
when fundamentals.dimensions
  process_fundamentals(incoming)
end
```

Reading `ohlcv.dimensions === incoming` as "does `incoming` have at least the dimensions `ohlcv` has?"

The template-match is a structural conformance check, not strict equality — `incoming` may have additional dimensions and still match. This matches Ruby's `===` convention (`Integer === 5` is true even though 5 is also a Numeric, a Comparable, and so on — match means "fits this category," not "exactly this and nothing else").

For exact-match semantics (the candidate has *exactly* these dimensions, no more), use `==`:

```
case incoming.dimensions
when ohlcv.dimensions then ...     # exact match via Array#==
when fundamentals.dimensions then ...
end
```

This works because both sides are `Namo::Dimensions` (subclasses of Array), and `===` between them falls through to `super` (Array#===, which delegates to ==).

Comparison through aspect projection

Because `coordinates[:dim]` and `values[:dim]` both return plain Ruby Arrays, the full vocabulary of Ruby's Array operators is available without Namo defining anything new:

```
a.coordinates[:symbol] == b.coordinates[:symbol]    # set equality on a dimension
a.values[:close] == b.values[:close]                # sequence equality on a dimension
a.coordinates[:date] <=> b.coordinates[:date]       # ordering on unique values
a.values[:close].sum                                # any Array method
namos.sort_by{|n| n.values[:date].first}            # sort Namos by aspect
```

This is the answer to "where does ordering live?" — at the aspect level, where projections to Arrays inherit `<=>` and `==` from Ruby's built-ins. No `Namo#<=>`, no Namo-level total order. Just well-chosen aspect methods exposing plain data structures.

Why Array storage, not Set

`values` reinforces the decision to keep `@data` as `Array<Hash>` rather than `Set<Hash>`. `values` needs ordering and duplicates — both things Set throws away. If the internal store were a Set, `values` would lose row order and couldn't contain duplicate rows. `to_a` would need to reconstruct an order that no longer exists internally. `to_h` (columnar) would have the same problem — the column arrays need a consistent row ordering so that `values[:symbol][i]` and `values[:close][i]` correspond to the same row. Set buys fast membership testing and automatic deduplication, but Namo doesn't need either of those as primitives. The set operations (`&`, `|`, `^`) work on Array storage just fine.

0.8.0: Proc-based and regex-based selection

Two ways to extend `[]` selection beyond exact values, arrays, and ranges: procs for arbitrary predicates, regexes for string pattern matching. Both are single-branch additions to existing selection logic, paired here because they share the same dispatch site (`Row#match?`).

Proc-based selection

Extend `[]` to accept procs as selection predicates on dimensions.

```
namo[pe: ->(v){ v && v < 15 }]
namo[price: ->(v){ v > 10.0 }, symbol: ->(v){ v != 'TEST' }]
```

Implementation: add a `when Proc` branch to `Row#match?` that calls the proc with the dimension value. Single addition to existing selection logic.

This enables multi-factor screening in one expression:

```
namo[pe: ->(v){ v && v < 15 }, price_to_book: ->(v){ v && v < 1.5 }]
```

Proc-based selection composes with contraction and projection in a single `[]` call.

Regex-based selection

Extend `[]` to accept regexes as selection predicates on string-valued dimensions.

```
namo[symbol: /^BH/]                    # symbols starting with BH
namo[symbol: /gold/i]                  # case-insensitive match
namo[sector: /mining|resources/i]      # alternatives
namo[symbol: /^BH/, sector: 'Energy']  # regex + exact, composable
```

Implementation: add a `when Regexp` branch to `Row#match?`:

```
when Regexp
  coordinate.match?(row[dimension].to_s)
```

Same weight as adding proc support — one additional `when` branch in the same `case` statement.

Regex is more ergonomic than the equivalent proc for string pattern matching:

```
# Regex
namo[symbol: /^BH/]

# Equivalent proc
namo[symbol: ->(v){ v.to_s =~ /^BH/ }]
```

The regex form is shorter, more declarative, and immediately legible. It doesn't replace procs — procs handle arbitrary logic — but for pattern matching on string-valued dimensions it's the natural tool.

Regex composes with all other selection types in the same `[]` call: exact values, arrays, ranges, and procs.

0.9.0: Composition operators (*, **, /)

The dimensional composition algebra.

* (equi-join on shared dimensions)

Identifies shared dimension names between two Namos. Pairs rows where coordinates match on all shared dimensions. Non-shared dimensions extend the result.

```
ohlcv * fundamentals  # joins on exchange, symbol
```

** (Cartesian product)

Every row from the left paired with every row from the right. No automatic matching. The "explosive" operator — more sigil, more output.

** without constraints produces the full Cartesian product. * is derivable from ** — it's ** with shared-dimension filtering applied automatically.

/ (decomposition)

The inverse of *. Factors out dimensions.

```
combined / ohlcv  # removes dimensions exclusive to ohlcv, keeps shared + fundamentals dimensions
```

`(combined / ohlcv) * ohlcv` reconstructs combined.

0.10.0: Blocks on comparison, composition, and set operators

All operators that match rows gain optional blocks that relax row equality from exact match to a custom predicate. The block-form pattern is consistent across the operator families that compose, set-combine, and compare.

Blocks on ==, <, <=, >, >=

Comparison operators with a block use the block as the row-matching predicate, replacing exact row equality.

```
# Equal as multisets when matched on symbol alone
a.==(b){|ra, rb| ra[:symbol] == rb[:symbol]}

# a's rows are a subset of b's, matching on symbol and date
a.<=(b){|ra, rb| ra[:symbol] == rb[:symbol] && ra[:date] == rb[:date]}
```

Useful when comparing Namos that should be considered "the same" on identifying dimensions even if other fields differ. A trading screen run on Monday and Tuesday with the same symbols-of-interest but different prices is `==` with a `{|ra, rb| ra[:symbol] == rb[:symbol]}` block, even though exact-row `==` returns false.

`eql?` does not take a block. Its purpose as the strictest level of equality (class + data + formulae) would be undermined by relaxation. The block forms are for the multiset-theoretic operators only.

Blocks on * and **

Both * and ** gain optional blocks for custom matching logic beyond exact dimension matching.

For *, the block receives a row from the left and the pre-filtered candidates from the right (already matched on shared dimensions):

```
ohlcv.*(fundamentals) do |row, candidates|
  candidates.select{|f| f[:quarter_end] <= row[:date]}.max_by{|f| f[:quarter_end]}
end
```

The block can be passed as a named proc:

```
MOST_RECENT_QUARTER = proc do |row, candidates|
  candidates.select{|f| f[:quarter_end] <= row[:date]}.max_by{|f| f[:quarter_end]}
end

ohlcv.*(fundamentals, &MOST_RECENT_QUARTER)
```

For **, the block receives a row and ALL rows from the right (no pre-filtering):

```
ohlcv.**(fundamentals) do |row, candidates|
  # full control, including symbol matching if desired
end
```

** with a block that manually matches on shared dimensions produces the same result as * without a block. * is sugar on top of **.

Blocks on set operators

All set operators (+, -, &, |, ^) gain optional blocks that relax the matching condition from exact row equality to a custom predicate.

```
# Remove by symbol match rather than exact row match
today.-(exclusions){|a, b| a[:symbol] == b[:symbol]}

# Add candidates not already held
```

```
portfolio.+(new_candidates){|existing, candidate| candidate[:symbol] != existing[:symbol]}
```

with a block uses the same semantics as + with a block, then deduplicates.

The unifying pattern

Every operator that conceptually pairs rows uses the same block contract: a two-argument block returning a boolean, called for each candidate pairing. This consistency means users learn the pattern once and apply it across operator families. The set, comparison, and composition operators all become tunable in the same way.

0.11.0: Two-arity formulae

Row gains a reference to its parent Namo. Procs with arity 2 receive (row, namo), enabling cross-row computation:

```
e[:sma_20] = proc do |row, namo|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(20)
  window.sum{|r| r[:close]} / window.length.to_f
end
```

The `each` method passes `self` (the Namo) to Row:

```
def each(&block)
  return enum_for(:each) unless block_given?
  @data.each{|row_data| block.call(Row.new(row_data, @formulae, self))}
end
```

Row's constructor gains an optional Namo reference:

```
class Row
  def initialize(row, formulae, namo = nil)
    @row = row
    @formulae = formulae
    @namo = namo
  end
end
```

The `namo` parameter defaults to `nil` for backward compatibility — existing code that calls `Row.new(row, formulae)` keeps working. Row-scoped formulae never touch `@namo`. Two-arity formulae use it when present. If a Row is constructed without a Namo and a two-arity formula is called, the `nil` produces a clear error rather than a missing argument error.

Note: the current Row constructor (0.5.0) takes `(row, formulae)`. Adding `@namo` is a prerequisite for two-arity formulae; it is introduced in this release. Subsequent releases that need the Namo reference (Finite for `last(n)` row wrapping in 0.13, parameterised formulae in 0.12) inherit the constructor change.

Row#[] dispatch for arity 1 and 2: - 1: `proc.call(row)` — row-scoped - 2: `proc.call(row, namo)` — collection-scoped

0.12.0: Parameterised formulae

Procs with arity > 2 receive (row, namo, *extra_args). Row#[] forwards extra arguments:

```
e[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(period)
  window.sum{|r| r[field]} / window.length.to_f
end

row[:sma, :close, 20]  # Row inserts self and namo, forwards :close and 20
```

Row#[] dispatch extended: - 1: `proc.call(row)` — row-scoped - 2: `proc.call(row, namo)` — collection-scoped - >2: `proc.call(row, namo, *extra_args)` — parameterised

0.13.0: Finite module

A new module that includes Enumerable and adds `last` and `reverse_each` for finite collections. Namo includes Finite instead of Enumerable. Default implementation uses `entries`; Namo overrides for performance by going straight to `@data`.

```
module Finite
  include Enumerable

  def last(n = nil)
    n ? entries.last(n) : entries.last
  end

  def reverse_each(&block)
    return enum_for(:reverse_each) unless block_given?
    entries.reverse_each(&block)
  end
end
```

Finite is a standalone concept — potentially a separate gem. Any finite Enumerable can include it.

Namo includes Finite and overrides `last` to go straight to `@data`, wrapping results in Rows. The Row constructor's `@namo` parameter (introduced in 0.11) is passed through so Finite-wrapped Rows participate in two-arity formulae just like rows from `each`:

```
def last(n = nil)
  if n
    @data.last(n).map{|row| Row.new(row, @formulae, self)}
  else
    Row.new(@data.last, @formulae, self)
  end
end
```

0.14.0: Enumerable methods return Namos

Enumerable methods that return a subset or reordering of rows should return a Namo, not a raw Array of Rows. Currently `namo.select{|row| row[:close] > 40.0}` returns an Array, requiring a manual `Namo.new(...)` to continue working with the result as a Namo. This is ceremony — you selected rows from a Namo, you expect a Namo back.

These are sequence-view operations: `select`, `reject`, `sort_by`, `first(n)`, `last(n)` all care about row order and produce ordered subsets. They sit alongside the set-view operators (`==`, `<`, `&`, `|`, etc.) introduced in 0.4.0–0.6.0. Namo's dual nature — set when membership is what matters, sequence when order is what matters — is realised across both families: set operators ignore order and produce set-correct results; Enumerable methods respect order and produce ordered results. The same Namo supports both views.

```
# Before (0.2.0-0.13.0)
filtered = namo.select{|row| row[:close] > 40.0}
filtered.class             # => Array
filtered[symbol: 'BHP']   # => NoMethodError
filtered = Namo.new(filtered)
filtered[symbol: 'BHP']   # works

# After (0.14.0)
```

```
filtered = namo.select{|row| row[:close] > 40.0}
filtered.class              # => Namo
filtered[symbol: 'BHP']   # works - selection, projection, formulae, everything
```

Methods that should return Namos: `select`, `reject`, `sort_by`, `first(n)`, `last(n)`. These produce ordered subsets of rows — still a Namo.

Methods that should not: `map`, `reduce`, `sum`, `min_by`, `max_by`, `flat_map`, `each`. Their return types are genuinely different from a Namo — scalars, transformed values, enumerators.

`group_by` is also excluded from 0.14.0 — its return type is `{key => Array<Row>}`, a hash of subsets rather than a single subset. 2.x revisits this and wraps each group's array in `Namo.new`, making `group_by` return `{key => Namo}`. The 0.14.0 decision is therefore an interim one, settled here at the level the simple wrapping pattern can handle and revisited when the bare-name and richer-typing work of 2.x makes the further enrichment natural.

Implementation: override the subset-returning methods to wrap the result in `Namo.new`, carrying formulae through.

1.0.0: Stable release

The 1.0 release includes everything through 0.14.0: comparisons, aspect classes (`Namo::Dimensions`, `Namo::Coordinates`, `Namo::Values`) with template-match `===`, `values` and `to_h`, proc-based and regex-based selection, composition operators (`*`, `**`, `/`), blocks on all operators, two-arity formulae, parameterised formulae, Finite module, and Enumerable methods returning Namos. This is the correct, tested, conservative foundation. No metaprogramming magic, no `method_missing`, no `instance_eval`. Formulae work via `e[:name] = proc{|row| row[:close] / row[:book_value]}` — clear, explicit, proven.

1.0 is the set of features that are well-understood, thoroughly tested, and unlikely to change.

Estimated performance: ~0.3s for a 2,000 row daily trading screen with indicators and scoring. Pure Ruby, no native dependencies. Adequate for interactive use and daily batch jobs. Not suitable for datasets over ~50,000 rows without patience.

1.1: Benchmarking suite

Performance and stress-testing infrastructure. Establishes baselines for every feature so that subsequent optimisations can be measured against concrete numbers.

The suite should cover:

- Construction: `Namo.new` from N rows, measuring hash ingestion and dimension/coordinate inference
- Selection: single value, array, range, proc, regex, at various dataset sizes
- Projection and contraction: `[]` dispatch overhead
- Formulae: row-scoped resolution, formula chains (A references B references C), parameterised formulae
- Enumerable: `each`, `map`, `select`, `reduce`, `max_by` — measuring Row object creation per iteration
- Set operators: `+`, `-`, `&`, `|`, `^` at various dataset sizes
- Composition: `*` across different dimension overlaps

Each benchmark at multiple scales — 100, 1,000, 10,000, 100,000, 1,000,000 rows — to show where curves bend and where Ruby's overhead dominates.

Use `benchmark-ips` for iterations-per-second measurements. Results should be recorded and versioned so regressions are visible across releases.

The 1.1 numbers become the baseline for 2.x comparisons.

1.2: Loaders

Optional requires for common data sources. No loader loaded by default. No dependencies added to Namo core.

```
require 'namo/loaders/csv'
require 'namo/loaders/sequel'
require 'namo/loaders/json'
```

Each extends Namo with a class method (`from_csv`, `from_sequel`, `from_json`). Loaders are sugar — the constructor already takes arrays of hashes, so loading is always possible without them.

2.x: Bare names and Ruby-side optimisation

Theme: the expressive leap.

Estimated performance: `method_missing` on Row adds ~5-10% overhead versus 1.x on first access per dimension per Row. The `DefineAccessors` optimisation (below) recovers most of this by defining real methods on first access. After warm-up, 2.x should be within ~2-3% of 1.x. Pure Ruby performance tuning against the 1.1 benchmarking suite may yield further gains. All estimates — actual numbers depend on the 1.1 baseline.

From 2.0, `require 'namo'` loads bare name resolution by default. The full experience is the default. Users who want the 1.x explicit style can opt out:

```
require 'namo'          # 2.x default: bare names included
require 'namo/core'     # opt out: 1.x explicit style, no method_missing on Row
```

Bare name resolution

Row resolves dimension names and formulae as bare method calls via `method_missing`, delegating to `self[name, *args]`. This eliminates hash access syntax from formulae:

```
# 1.x style
e[:price_to_book] = proc{|row| row[:close] / row[:book_value]}

# 2.x with bare names via `method_missing` on Row
e[:price_to_book] = proc{|row| row.close / row.book_value}

# 2.x with bare names via `instance_eval` for arity-0 procs
e[:price_to_book] = proc{ close / book_value }
```

Bare names also work in cross-row formulae. Fixed dimension references resolve against the current Row. Variable field names (passed as parameters) use `Symbol#to_proc` via `&field`:

```
e[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: symbol, date: ..date].last(period)
  window.sum(&field) / window.length.to_f
end
```

Here `symbol` and `date` are bare names resolving against the Row. `&field` uses `Symbol#to_proc` to send the field name to each Row in the window.

Resolution order in `Row#method_missing`: formulae first, then row data, then `super` (raises `NoMethodError` for typos).

`method_missing` on Namo for formula assignment

Namo resolves formula assignment as bare method calls via `method_missing`, catching `name=` calls:

```
e.price_to_book = proc{ close / book_value }
e.sma = proc do |row, namo, field, period|
  # ...
end
```

Module-based formula libraries (documented pattern)

With bare name resolution in place, formulae can be defined as plain Ruby methods in modules and included into Namo subclasses. This is not a Namo feature — it's standard Ruby. It's documented here as a recommended pattern:

```
module Indicators
  def sma(row, namo, field, period)
    window = namo[symbol: symbol, date: ..date].last(period)
    window.sum(&field) / window.length.to_f
  end

  def change
    close - open
  end

  def earnings_yield
    eps / close
  end
end

module Scoring
  def value_score
    return 0 unless pe && pe > 0
    pe < 10 ? 2 : pe < 15 ? 1 : pe < 25 ? 0 : -1
  end

  def total_score
    value_score + book_score + yield_score + momentum_score + trend_score
  end

  def action
    s = total_score
    s >= 5 ? 'BUY' : s >= 2 ? 'WATCH' : s >= 0 ? 'HOLD' : total_score <= -2 ? 'SELL' : 'AVOID'
  end
end

class TradingAnalysis < Namo
  include Indicators
  include Scoring
end
```

Scoring strategies are swappable via dependency injection.

`Namo#===` revision under module-based formulae

When module-based formulae become a real pattern (rather than a documented one), `Namo#===` (and `Namo#eql?`, and `Namo#hash`) must be revised to enumerate module-defined method names alongside `@formulae.keys`. The 0.6.0 implementation compares formula *names* via `@formulae.keys.sort` — which is correct for `[]=`-registered formulae, but module-defined formulae don't enter `@formulae`. They're plain

Ruby methods on the included module, discovered by Row's `method_missing` through the normal method-lookup chain. So a `TradingAnalysis` instance with `include Indicators` would have `@formulae.keys == []` even though it has the full `Indicators` analytical surface; under 0.6.0's `===`, two such instances are correctly `===` to each other (same dimensions, same — empty — `@formulae.keys`), but they'd also be `===` to a vanilla `Namo` of the same dimensions, which is wrong: a vanilla `Namo` has no indicators.

The starting point

```
# 0.6.0 implementation
def ===(other)
  return false unless other.is_a?(Namo)
  dimensions.sort == other.dimensions.sort &&
    @formulae.keys.sort == other.formulae.keys.sort
end
```

Correct for the case where all analytical structure flows through `@formulae`. The gap is everything that lives on the class instead of the instance.

Alternatives considered Three candidates were on the table:

Path A: Check class identity. `===` also requires `self.class == other.class`, matching `eql?`'s class-strictness. Same class implies same included modules implies same module-defined formulae, by transitivity. Simple to implement but rejects useful equivalences — two Namos from different classes with the same queryable surface (one via `[]=`, one via `include`) would not be `===` even though they behave identically as analytical artefacts.

Path B: Duck-type the queryable surface. `===` compares the full set of names queryable on each Namo, regardless of where each name's definition lives. Storage dimensions, instance `@formulae` keys, and class-level analytical method names all contribute. Two Namos with the same queryable surface are `===` regardless of class or how their formulae were registered. Matches the design philosophy's unified-treatment principle: care about what's queryable, not where the definition lives.

Path C: Class as proxy for analytical surface. Same code as Path A, framed differently — class identity stands in for the union of all analytical methods. This is just Path A with different rationale; the behaviour is identical and the same objection applies.

Path B is the right answer. It honours duck-typing across class boundaries, handles the `[]=`-vs-`include` mixture correctly, and doesn't change behaviour when users refactor a formula from per-instance assignment to a module method.

Path B implementation sketch

```
def ===(other)
  return false unless other.is_a?(Namo)
  queryable_names == other.send(:queryable_names)
end

private

def queryable_names
  (dimensions + @formulae.keys + class_formula_names).to_set
end
```

`to_set` gives order-insensitive comparison; `.sort` would also work and avoids the Set allocation. Either is fine — pick when implementing. The equivalent changes to `eql?` and `hash` follow the same shape: substitute `queryable_names` for `@formulae.keys.sort` everywhere they appear.

What `class_formula_names` returns The open design question. Two candidate mechanisms:

Implicit identification. All public instance methods on the Namo subclass that aren't inherited from `Namo` itself are treated as formulae. Simple, no declaration overhead. Cost: conflates analytical methods with any custom method — a `def to_s` override on the subclass would be misread as a formula.

Explicit declaration. Modules opt into being treated as formula libraries, either by a marker module (`include Formula::Module`) or per-method declaration (`formula :sma`). Cost: declaration overhead. Benefit: precise — only intentionally-analytical methods count.

The implicit approach is more ergonomic; the explicit approach is more robust. Choose based on what the module-based formula mechanism actually looks like once it's implemented.

Body equality is not part of any equality operator 0.6.0 already settled this for the instance-level case: `===`, `eql?`, and `hash` all compare formula *names* (`@formulae.keys.sort`), not the procs themselves. The Path B revision extends that stance to module-defined formulae — adding more names to compare, not adding proc-body comparison. The rationale is the same as at 0.6.0 design time:

1. **Proc identity isn't function identity.** Two interactively-built Namos defining `:doubled = proc{|r| r[:x] * 2}` separately have different proc objects. Comparing by proc identity would call them unequal despite identical behaviour. Users would be surprised.
2. **`===` is a pattern-match operator, not a behaviour-identity operator.** Pattern-matching asks "does this candidate fit this pattern?" The pattern is the analytical shape — what names are queryable. `eql?` makes the same call for the same reason: in Ruby there's no cheap, reliable way to compare two procs for behavioural equivalence, so neither operator pretends to.

The deeper question of true behavioural equivalence (AST comparison, iseq comparison, DSL normal forms, etc.) is its own significant project; see the "Proc equivalence options" note in the project memory for the landscape. Until one of those paths lands, key-based comparison is the honest pragmatic position.

When this revision lands In whichever 2.x release introduces module-based formula registration as a real mechanism (rather than just a documented pattern using plain Ruby methods). Until then, the 0.6.0 key-based implementation handles `[]=`-registered formulae correctly; the gap (module-defined methods) only matters once those methods actually exist as a registration channel.

`DefineAccessors` optimisation

Inspired by Hashie's pattern — lazily define real methods on first access to avoid repeated `method_missing` overhead:

```
class Row
  def method_missing(name, *args)
    if @formulae.key?(name)
      define_singleton_method(name) do |*a|
        resolve(name, *a)
      end
      send(name, *args)
    elsif @row.key?(name)
      define_singleton_method(name) { @row[name] }
      @row[name]
    else
      super
    end
  end
end
```

First access fires `method_missing` and defines a real method. Every subsequent access is a direct method call. Consider when profiling shows `method_missing` as a bottleneck.

Hashie as prior art, not a dependency

Hashie (particularly Hashie::Mash) is the primary prior art for method-style hash access and coercion in Ruby. Namo's Row and the coercion module design draw from Hashie's patterns. However, Hashie should not be used as a dependency or as internal storage for Namo:

- Namo already handles method-style access through Row's `method_missing`. Wrapping internal hashes in Hashie::Mash would add a second `method_missing` layer — double the dispatch overhead for no additional capability.
- Hashie does key normalisation (string/symbol indifference) and deep conversion of nested hashes. Namo doesn't need either — keys are always symbols and data should be flat.
- Hashie has a documented history of subtle breakage, particularly around `respond_to_missing?` interactions with other gems and method name collisions with Ruby built-ins.
- The performance cost of Mash allocation over plain Hash allocation is measurable.

Pure Ruby performance tuning

Profile against the 1.1 benchmarking suite. Identify and address hot paths within pure Ruby: Row object allocation, formula chain resolution, selection dispatch. No native code — just better Ruby.

Group-by returns Namos

`Enumerable#group_by` currently returns `{key => Array<Row>}` (per 0.14.0 — the explicit decision there is that `group_by`'s return type is "genuinely different from a Namo" and therefore stays as a hash of arrays). 2.x revisits that decision: each group becomes a Namo, retaining the parent's formulae and supporting the full Namo vocabulary on the result.

```
# 1.x
namo.group_by{|r| r[:symbol]}
# => {'BHP' => [<Row>, <Row>, ...], 'RIO' => [...]}

# 2.x
namo.group_by{|r| r[:symbol]}
# => {'BHP' => <Namo>, 'RIO' => <Namo>}
```

Aggregation pipelines become Namo-native:

```
# 2.x - each group is a queryable Namo
namo.group_by{|r| r[:symbol]}
  .transform_values{|n| n.values[:close].sum / n.length}
# => {'BHP' => 42.8, 'RIO' => 118.3}

# Or using bare names (also 2.x):
namo.group_by(&:symbol)
  .transform_values{|n| n.close.sum / n.length}
```

The shift is small in implementation (wrap each group's array in `Namo.new`, carrying formulae through) but significant in user experience: `group_by` joins `select`, `reject`, `sort_by`, `first(n)`, and `last(n)` as Enumerable methods that produce Namos. The conceptual model unifies — every Enumerable-derived subset of a Namo's rows is itself a Namo.

Why 2.x rather than 0.14.0? Two reasons. First, it's a richer change than the simple "wrap subset returns" pattern of 0.14.0 — `group_by` returns a Hash of subsets, not a single subset, and the design needs to settle what state each group's Namo carries. Second, it pairs naturally with bare name resolution: `n.close.sum` only reads as ergonomically as it does once bare names are available, and bare names are a 2.x feature.

Open question for the design: when a group's Namo is created, what does its `coordinates` look like? If the parent has `coordinates[:symbol] = ['BHP', 'RIO', 'CBA']` and we group by `:symbol`, the `'BHP'` group's Namo has only BHP rows. `coordinates[:symbol]` on that group could be `['BHP']` (filtered to what's actually present) or `['BHP', 'RIO', 'CBA']` (inherited from parent). Filtered is probably right — the group is a fresh Namo, and its coordinates should reflect its own contents — but this needs a design pass before implementation.

Finite as a separate gem

Extract the Finite module (introduced in 0.13.0) into a standalone gem for use by any finite Enumerable, independent of Namo.

3.x: DSL, columnar storage, and C acceleration

Theme: the optimised engine.

Estimated performance: columnar storage accelerates selection-heavy workloads — scanning one contiguous array versus touching every row's hash. C extension via RubyInline targets the remaining hot paths (iteration, row creation). Combined, the 2,000 row daily screen should drop from ~0.3s to ~0.1s. Selection on large datasets (100,000+ rows) should see the biggest improvement from columnar storage. The C extension adds ~4x speedup on tight numeric loops but the bottleneck is object allocation, not arithmetic — expect modest overall gains from C alone. All estimates.

From 3.0, `require 'namo'` loads both bare names and the DSL block syntax by default. Users can opt out at any granularity:

```
require 'namo'              # 3.x default: bare names + DSL
require 'namo/core'         # 1.x explicit style only
require 'namo/bare_names'   # bare names without DSL
require 'namo/dsl'          # DSL block syntax without bare names
require 'namo/dsl/bare_names'  # both (same as default, explicit about it)
```

The DSL and bare names are independent features on different objects — the DSL uses `method_missing` on a builder to catch formula registrations, while bare names use `method_missing` on Row to resolve dimension access. The DSL block works without bare names (procs use explicit `row[:close]` inside), and bare names work without the DSL block (formulae registered via `e.name = proc{}`). In practice, most users will want both.

DSL block syntax

A minimal-ceremony specification syntax using a builder with `method_missing`:

```
namo do
  sma              ->(field, period) { window(field, period).mean }
  change           -> { close - open }
  price_to_book    -> { close / book_value }
  earnings_yield   -> { eps / close }
  golden_cross     -> { sma(:close, 20) > sma(:close, 50) }
  value_score      -> { pe < 10 ? 2 : pe < 15 ? 1 : pe < 25 ? 0 : -1 }
  total_score      -> { value_score + book_score + yield_score + momentum_score + trend_score }
  action           -> { total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WATCH' : total_score >= 0 ? 'HOL
end
```

The builder's `method_missing` catches each name-lambda pair. No `def`, no `end`, no `module`, no `class`, no `e.name = proc`. Just names and computations.

Columnar storage

A columnar backend (hash of arrays) to accelerate selection and aggregation for pure-Ruby workloads. The design uses a mezzanine pattern — `Namo` delegates to `Namo::RowStore` or `Namo::ColumnStore` with the same public API.

This section is about *internal storage*, not about adding new public methods. `to_h` and `values` (both producing columnar output) already exist from 0.7.0; what 3.x changes is the cost of computing them. With row-oriented storage (current), `to_h` builds the columnar hash on demand by iterating rows. With columnar storage, `to_h` returns the internal representation directly. Same output, different cost.

The columnar form is exactly the shape that `to_h`/`values` produces — `{symbol: ['BHP', ...], close: [42.5, ...]}`, a hash of arrays. Columnar storage means accepting that shape as input and using it as the internal representation. The round-trip is clean: `Namo.new(namo.to_h)` reconstructs from columnar output.

Development order:

1. Default (current): always RowStore, no choice, no configuration
2. Choose at instantiation: Namo detects the input shape (array of hashes vs hash of arrays) or accepts an explicit `storage:` keyword
3. Dynamic: one primary layout with the other cached on demand, cache invalidated on mutation

```
# Row-oriented (array of hashes, current)
namo = Namo.new([{symbol: 'BHP', close: 42.5}, ...])

# Column-oriented (hash of arrays) - the shape to_h produces
namo = Namo.new({symbol: ['BHP', ...], close: [42.5, ...]})
```

When DuckDB is the backend (4.x), neither layout matters — the data lives in DuckDB. Columnar storage only matters when Namo is doing the computation itself in pure Ruby.

C extension via RubyInline

Optional C acceleration for hot paths via RubyInline (zenspider). Targets: selection (linear scan over contiguous array), iteration (row creation), and storage layout. Formula resolution stays in Ruby.

Pure Ruby is the default. C extension is an optional `require`. The API doesn't change:

```
require 'namo'
require 'namo/ext'  # optional C acceleration, falls back to pure Ruby if unavailable
```

Coercion modules

Two forms of coercion, both declared as includable modules:

Ingestion coercion — fix types from CSV/JSON:

```
module OHLCVTypes
  def self.included(base)
    base.coerce :date, Date
    base.coerce :close, Float
    base.coerce :volume, Integer
  end
end
```

Dimension alignment coercion — map dimensions for `*` composition:

```
module TemporalAlignment
  def self.included(base)
    base.coerce :date, to: :quarter do |date|
      "#{date.year}-Q#{((date.month - 1) / 3) + 1}"
```

```
    end
  end
end
```

Coercion modules can be placed on the analysis class, on a loader, or on an ad hoc instance. Namo doesn't have an opinion about where they live. Inspired by Hashie's coercion extensions, but applied at the dimensional level.

The two forms are distinguishable by signature: - `coerce :date, Date` — ingestion, fix the type of this dimension's values - `coerce :date, to: :quarter do ... end` — alignment, map this dimension to another for `*`

Nested data flattening

Data from JSON APIs and other sources may arrive with nested structures:

```
{symbol: 'BHP', fundamentals: {eps: 1.2, pe: 14.5}}
```

Nested values break Namo's foundational principle — every key is a dimension. The solution is flattening at ingestion via a coercion declaration:

```
base.coerce :fundamentals, extract: [:eps, :pe, :book_value]
```

Which pulls specified keys up to the top level and drops the wrapper. The result is a flat hash where everything is a dimension.

Conversion discovery on *

When `*` finds no exact dimension name match, it checks the class's coercion registry for declared conversion paths. This replaces the earlier `respond_to?` design with explicit, inspectable, testable declarations.

Co-incident: Python bridge Paths 1 and 2 (separate repo)

Developed in a separate `namo-python` repository. The Python user writes Namo's Ruby DSL inside a triple-quoted string. The 3.x DSL block syntax is what they see:

```
from namo import Namo

analysis = Namo.define("""
  e.golden_cross   = proc{ sma(:close, 20) > sma(:close, 50) }
  e.earnings_yield = proc{ eps / close }
  e.action         = proc{ total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WATCH' : 'HOLD' }
""")

results = analysis.run(data)
```

Path 1 (shell out): Python wrapper writes the DSL to a temp file, invokes `ruby` via `subprocess`, parses JSON results from stdout. Requires separate Ruby installation. ~0.5s overhead per invocation.

Path 2 (`rb_call`): MessagePack RPC to a persistent Ruby process. Lower latency (~50ms per call), more complex setup. Still requires Ruby installed separately.

Both paths build against 3.x Ruby Namo. The Python user experiences the full expressiveness of the DSL syntax. Ships with the `Indicators` module as a freebie so Python users get immediate value.

4.x: mRuby, SQL pushdown, and DuckDB

Theme: the database-integrated engine.

Estimated performance: DuckDB pushdown is the architectural win. Window functions (SMA, EMA) on millions of rows execute at ~0.3s in DuckDB versus ~45s in pure Ruby. The 2,000 row daily screen drops to

~0.05-0.1s total — DuckDB handles the heavy computation, Ruby handles the scoring and selection on the small result set. For the full 8.8M row ASX history, DuckDB makes it feasible where pure Ruby never could. Competitive with Polars (~0.1s) on equivalent workloads. All estimates.

mRuby compatibility

Namo's core compiled under mRuby. This enables embedding Namo in other environments without a full CRuby installation. Connects to existing mRuby compatibility work (e.g. `stateful.rb` 2.1.0).

DuckDB integration

DuckDB as a high-performance backend via Sequel or the `duckdb` gem. Four paths were considered, ordered from most manual to most automatic:

Path A (rejected): Write raw SQL for large slabs of functionality. Rejected — means maintaining SQL alongside Ruby, and the two representations can drift apart.

Path D (immediate): Use Sequel to generate DuckDB window functions before Namo ingests the data. No changes to Namo.

Path C (next): Formulae carry SQL metadata alongside the Ruby implementation. Namo detects the backend and inlines SQL where available:

```
module Indicators
  def sma(row, namo, field, period)
    window = namo[symbol: symbol, date: ..date].last(period)
    window.sum(&field) / window.length.to_f
  end

  SQL = {
    sma: ->(field, period) {
      "AVG(#{field}) OVER (PARTITION BY symbol ORDER BY date ROWS BETWEEN #{period - 1} PRECEDING AND CU
    }
  }
end
```

Path B (eventually): Namo generates SQL from formula definitions automatically.

Different Namos in the same composition can use different backends. OHLCV from DuckDB with SQL pushdown, fundamentals from Sequel, macro data from CSV. They compose via `*` regardless of origin.

SQL pushdown from formula metadata

Formulae declare their SQL equivalents. Namo uses the SQL version when a database backend is detected and falls back to Ruby when it can't. The Ruby implementation is the truth. The SQL is the optimisation. Both are declared in the same place.

Co-incident: Python bridge Path 3 (separate repo)

With mRuby compatibility in 4.x, the Python bridge can embed the mRuby interpreter directly. `pip install namo` works with no separate Ruby dependency. Built in the `namo-python` repo against the 4.x mRuby-compatible core.

Near-zero call overhead — in-process, no serialisation. Cross-platform binary distribution needed (wheels per platform).

5.x: Streaming, transpilation, and genetic algorithms

Theme: the analytical platform.

Estimated performance: streaming eliminates the memory constraint — 8.8M rows never loaded simultaneously. Processing cost is per-window rather than per-dataset. Transpilation to SQL or Python/Polars delegates execution entirely to native engines — Namo becomes a specification tool with near-zero runtime cost. GA support benefits from DuckDB's speed on each candidate evaluation. Performance is bounded by the backend, not by Namo. All estimates.

Streaming / progressive computation

For large datasets (8.8M+ rows), process data in windows rather than loading everything. Rolling buffer per symbol, incremental indicator computation, results retained, raw data discarded.

A streaming Namo stays unfrozen — it accepts new rows, invalidates caches on mutation, and reflects current state. The analytical case is a Namo that gets frozen after setup.

Transpilation

Namo as a specification language that emits executable code in other targets:

- SQL (DuckDB, PostgreSQL) for production deployment
- Python/Polars for handoff to Python teams

Explore interactively in Ruby. Deploy as transpiled output. The exploration tool and the production artifact are the same object viewed from different angles.

Genetic algorithm support

Parameterised formulae enable GA-based strategy optimisation:

```
namo do |params|
  value_score -> { pe < params[:pe_strong] ? 2 : pe < params[:pe_good] ? 1 : 0 }
  action      -> { total_score >= params[:buy_threshold] ? 'BUY' : 'HOLD' }
end
```

Genome is a hash of parameters. Fitness is backtest return. Mutation/crossover operate on hashes. DuckDB evaluates each candidate at native speed.

Speculative / unversioned

These options are not scheduled. They become relevant when need and adoption justify the investment.

Rust acceleration via Rutie + Thermite

Rust accelerates specific hot paths within Ruby-side Namo. Ruby remains the host. The native code is an optimisation layer underneath the Ruby API. Formula resolution, `method_missing`, and DSL stay in Ruby.

Rutie provides Ruby bindings for Rust via macros. Thermite handles compilation and gem distribution with pre-compiled binaries. ~20x speedup observed on parsing benchmarks.

Targets: selection, iteration, storage layout. The same hot paths as the C extension (3.x) but with Rust's memory safety and the Rust ecosystem's momentum in data tooling (Polars, DuckDB, Apache Arrow).

```
namo/
  lib/
    namo.rb           # pure Ruby, always works
    namo/ext.rb        # loads native extension, falls back to pure Ruby
  ext/
```

```
  namo_ext/
    Cargo.toml       # Rust project
    src/
      lib.rs         # Rust implementations of hot paths
Rakefile             # Thermite tasks for building
```

Work on Rust acceleration builds familiarity and infrastructure that would feed into the Rust-native engine if it ever happens.

FFI (generic)

Compile any language (Rust, C, Crystal) as a C-compatible shared library (`cdylib`), load via Ruby's `ffi` gem or stdlib `Fiddle`. More boilerplate than Rutie but language-agnostic — the same FFI interface works regardless of what produced the shared library.

Rust-native engine with Ruby DSL parser

Namo's core reimplemented in Rust with a parser for the Ruby DSL syntax. The DSL specification strings are parsed, not evaluated — no Ruby runtime at all. The Rust engine is language-agnostic and can be exposed to any host language via its native FFI mechanism:

- Python via PyO3 (the same way Polars works)
- Ruby via Rutie (replacing pure Ruby internals with Rust for speed)
- Node.js via napi-rs
- Go via CGo
- Any language via C-compatible FFI

The Ruby DSL becomes a portable notation — a specification language that any host can submit to the Rust engine. The engine parses it, builds the internal representation, and executes it at native speed.

This path also connects to the transpilation story — if the Rust engine can parse the DSL, it can also emit SQL, Python/Polars expressions, or other output formats from the same parsed representation.

Not constrained to any single language. Largest engineering investment. Only justified if Namo achieves significant adoption across multiple language communities.

WASM compilation

Compile the Rust-native engine or mRuby core to WebAssembly for browser execution. Enables Namo-powered analysis in client-side web applications without a server.

Open questions

Mutability

Namo currently has `attr_accessor :data, :formulae`, making instances fully mutable. This conflicts with caching (stale results), thread safety (race conditions), and the principle that formulae are declarations.

The proposed model: Namos are mutable by default for interactive exploration. Call `freeze` when the shape is settled. Ruby's built-in `freeze` semantics apply — mutation methods (`[]=`, `.name=`) check `frozen?` and raise `FrozenError` if so. Caching is safe on frozen Namos. Threads can share frozen Namos.

```
namo = TradingAnalysis.new(ohlcv_data)
namo.change = proc{ close - open }
namo.score = proc{ ... }

# exploration done
namo.freeze
```

```
# now cacheable, shareable, immutable
namo.score = proc{ ... }  # => FrozenError
```

The streaming case is simply a Namo that never gets frozen. No special class needed — the distinction between analytical and streaming is a lifecycle difference, not a type difference. `freeze` is the transition point.

`attr_accessor` should become `attr_reader` on the public API, with mutation going through methods that check `frozen?`. Internal state (`@data`, `@formulae`) should be frozen when the Namo is frozen.

The `eql?`/`hash` contract introduced in 0.6.0 reinforces this. `eql?` requires `hash` to be consistent — `a.eql?(b)` implies `a.hash == b.hash` — and `hash` is computed from `@data`, class, and `@formulae`. Any mutation changes `hash`, which means an unfrozen Namo cannot be safely used as a Hash key or Set member: lookup might miss the entry the Namo was stored under because the Namo's hash has shifted. Frozen Namos have stable hashes and are safe for hash-keyed lookup. The lifecycle pattern (mutate during exploration, `freeze` before sharing) maps directly onto when `eql?`-based collection operations become reliable.

Live data and cache invalidation

For streaming/live Namos that stay unfrozen, formula results cached on Rows become stale when new data arrives. Options:

- No caching on unfrozen Namos (simple, slower)
- Generation counter — each mutation increments a counter, cached results carry the generation they were computed at, stale results are recomputed
- Event-based invalidation — mutation notifies dependents

The simplest approach (no caching when unfrozen) is probably right for the first implementation. Optimise if profiling shows it matters.

Coercion design

The coercion section (under 3.x) proposes a dual-purpose `coerce` keyword for both ingestion type fixing and dimensional alignment. Open questions:

- Is a single keyword for two different operations clear or confusing?
- Should ingestion coercion and alignment coercion be separate APIs?
- When multiple alignment coercions exist for a dimension (e.g. `:date` can coerce to both `:quarter` and `:month`), how does `*` choose? Raise ambiguity error? Accept a preference list?
- Should alignment coercion include a strategy (e.g. "most recent before") or just a value transformation?
- How does coercion interact with `freeze`? Are coercion declarations frozen with the Namo?
- Should nested data flattening (`extract:`) be part of the coercion API or a separate ingestion concern?
- Should flattening be recursive (nested hashes within nested hashes) or single-level only?

Formula shadowing and method collisions

If a formula and a data dimension share the same name, which wins in Row resolution? Currently formulae take priority. Should this be configurable? Should it warn?

Related: Hashie::Mash's long history of problems with method names that collide with Ruby built-ins (`class`, `hash`, `sort`, `zip`, `count`) is directly relevant. Row's `method_missing` has the same risk — a dimension named `class` or `method` would shadow Ruby's built-in methods. Options:

- Maintain a safelist of Ruby method names that `method_missing` won't intercept
- Warn when a dimension name collides with a built-in
- Always allow hash-style access (`row[:class]`) as a fallback when bare name access is blocked
- Document the risk and accept it — dimension names like `class` and `method` are unlikely in practice

Operator return types with subclasses

When `TradingAnalysis * Namo` is evaluated, what class is the result? `TradingAnalysis` (preserving the included modules)? `Namo` (the base class)? The left operand's class? This affects whether formulae from included modules are available on the composed result.

0.6.0 settles part of this question for equality: `eql?` cares about class match (`TradingAnalysis.new(data).eql?(Namo.new(d` returns false even if the data matches), `==` does not. The composition operators (`*`, `**`, `/`) and set operators (`+`, `-`, `&`, `|`, `^`) still need a settled answer — composed results currently default to the receiver's class, but cross-class composition (`TradingAnalysis * SectorMetrics`) raises the question of which subclass's modules carry through.

Presentation examples

See EXAMPLES.md for full four-stage progressions (competitor tool → 1.x → 2.x → 3.x) across seven disciplines with side-by-side code comparisons.